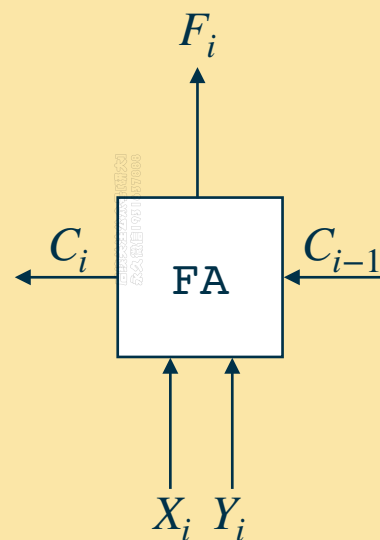




1. 基本运算部件

1.1 串行进位加法器

全加器用来实现两个本位数加上低位进位生成一位本位以及一位向高位的进位



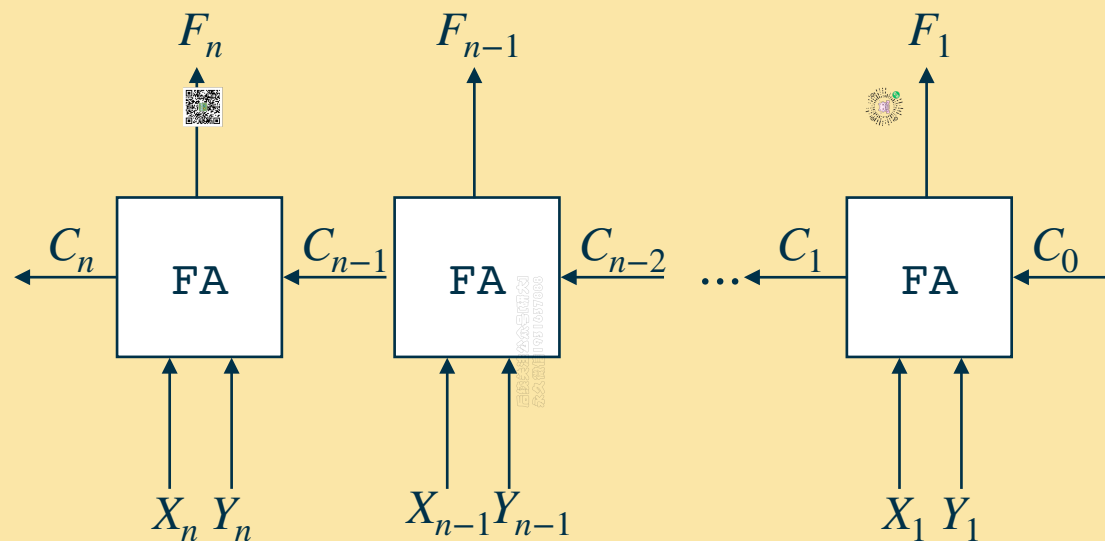
$$\text{和: } S_i = X_i \oplus Y_i \oplus C_{i-1}$$

$$\text{进位: } C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$$



1. 基本运算部件

1.1 串行进位加法器



n位串行进位加法器

将n个全加器相连得到n位串行进位加法器
x和y逐位相加，位间进位串行传送

$$\text{和: } S_i = X_i \oplus Y_i \oplus C_{i-1}$$

$$\text{进位: } C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$$

全加器用来实现两个本位数加上低位进位生成一位本位以及一位向高位的进位



1. 基本运算部件

1.2 并行进位加法器

$$\text{和: } S_i = X_i \oplus Y_i \oplus C_{i-1}$$

$$\text{进位: } C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$$

$$C_1 = X_1 Y_1 + (X_1 + Y_1) C_0$$

$$\begin{aligned} C_2 &= X_2 Y_2 + (X_2 + Y_2) C_1 \\ &= X_2 Y_2 + (X_2 + Y_2) X_1 Y_1 + (X_2 + Y_2) (X_1 + Y_1) C_0 \end{aligned}$$

$$\begin{aligned} C_3 &= X_3 Y_3 + (X_3 + Y_3) C_2 \\ &= X_3 Y_3 + (X_3 + Y_3) [X_2 Y_2 + (X_2 + Y_2) X_1 Y_1 + (X_2 + Y_2) (X_1 + Y_1) C_0] \\ &= X_3 Y_3 + (X_3 + Y_3) X_2 Y_2 + (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 \\ &\quad + (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= X_4 Y_4 + (X_4 + Y_4) C_3 \\ &= X_4 Y_4 + (X_4 + Y_4) [X_3 Y_3 + (X_3 + Y_3) X_2 Y_2 + (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 \\ &\quad + (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0] \\ &= X_4 Y_4 + (X_4 + Y_4) X_3 Y_3 + (X_4 + Y_4) (X_3 + Y_3) X_2 Y_2 \\ &\quad + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 \\ &\quad + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0 \end{aligned}$$



$$\text{和: } S_i = X_i \oplus Y_i \oplus C_{i-1}$$

$$\text{进位: } C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$$

1. 基本运算部件

1.2 并行进位加法器

$$C_1 = X_1 Y_1 + (X_1 + Y_1) C_0$$



$$C_2 = X_2 Y_2 + (X_2 + Y_2) C_1$$

$$C_1 = G_1 + P_1 C_0$$

C_3

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 C_0$$

图灵社区会员
童樱自1010101000

$$C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0$$

C_4

$$C_4 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$$

$$\begin{aligned} &= X_4 Y_4 + (X_4 + Y_4) X_3 Y_3 + (X_4 + Y_4) (X_3 + Y_3) X_2 Y_2 \\ &\quad + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 \\ &\quad + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0 \end{aligned}$$

每个进位表达式都含有 $X_i + Y_i$ 和 $X_i Y_i$

$X_i + Y_i$ 为 P_i

$X_i Y_i$ 为 G_i

每个进位表达式都含有 X_i+Y_i 和 X_iY_i



X_i+Y_i 为 P_i

X_iY_i 为 G_i

1. 基本运算部件

1.2 并行进位加法器

$$C_1 = G_1 + P_1 C_0$$

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 C_0$$

$$C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0$$

$$C_4 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$$

C_i 只与低位的 x 、 y 以及初始进位 C_0 有关

即，相互间的进位之间是没有依赖关系的

只要 $x_1 \sim x_4$ ， $y_1 \sim y_4$ 和 C_0 同时到达，就可几乎同时形成 $C_1 \sim C_4$ ，并同时生成各位的和



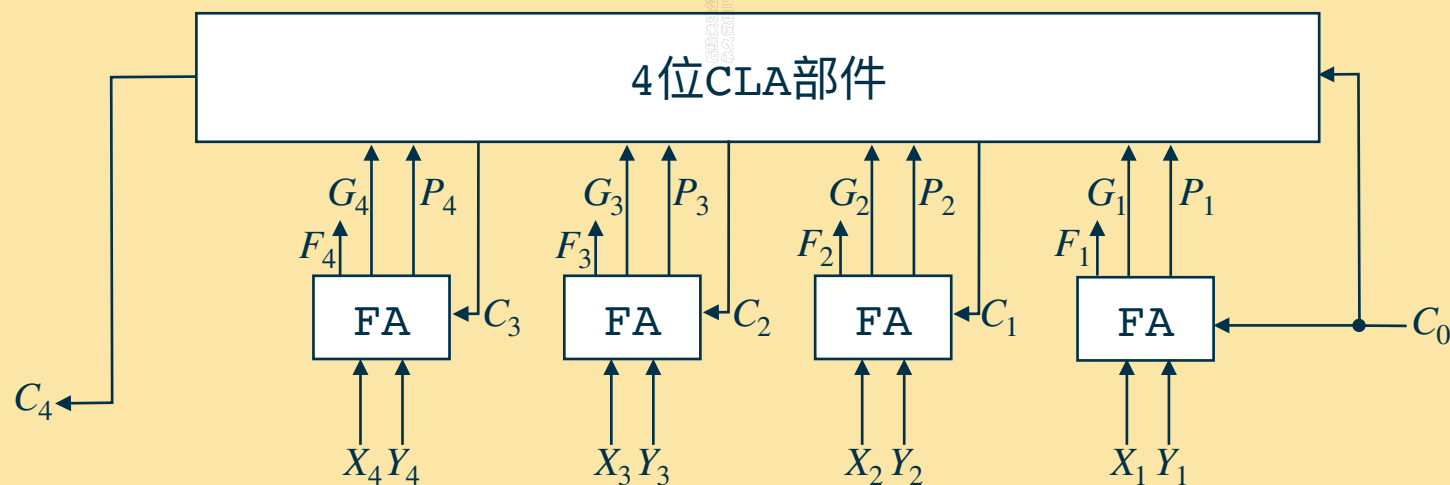
1. 基本运算部件

1.2 并行进位加法器

c_i 仅与 x_i 、 y_i 和 c_0 有关

即，相互间的进位之间是没有依赖关系的

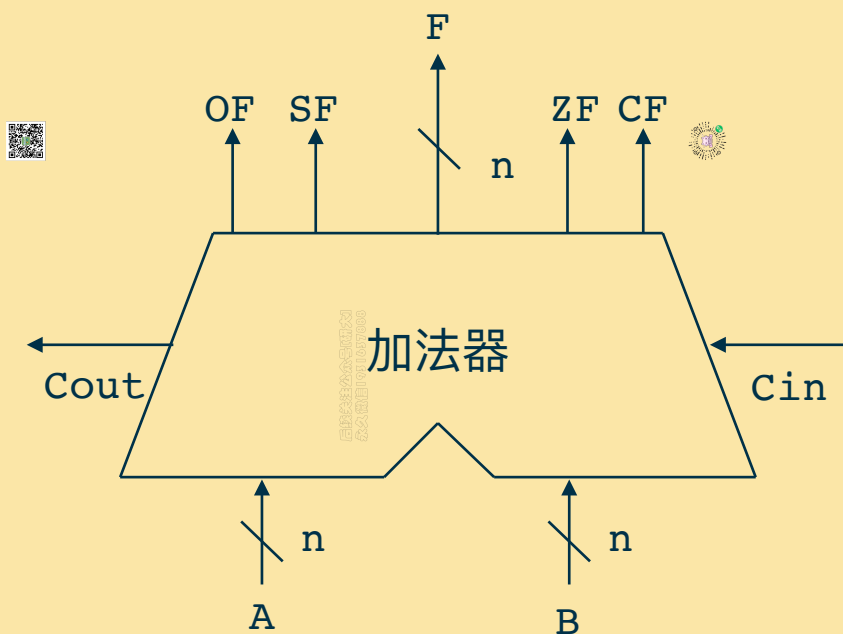
只要 $x_1 \sim x_4$ ， $y_1 \sim y_4$ 和 c_0 同时到达，就可几乎同时形成 $c_1 \sim c_4$ ，并同时生成各位的和





1. 基本运算部件

1.3 带标志加法器



带标志加法器符号

溢出标志 $OF = C_n \oplus C_{n-1}$

符号标志 $SF = F_{n-1}$

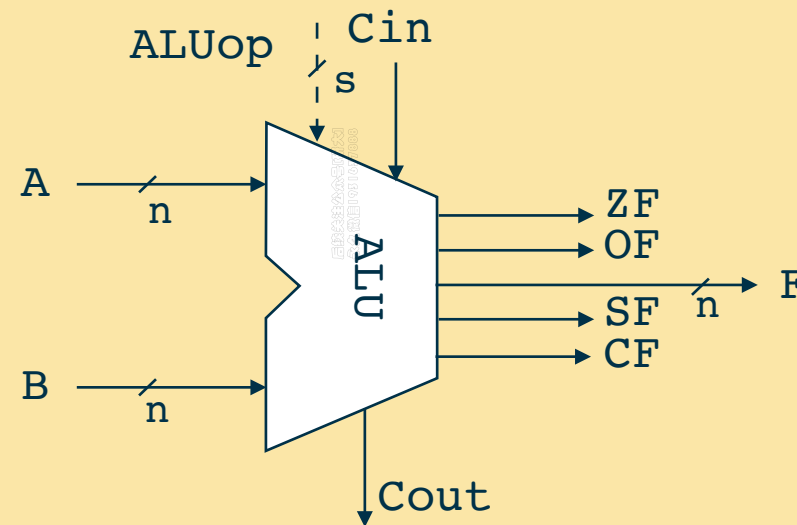
零标志 $ZF = 1$ 当且仅当 $F = 0$

进位/借位标志 $CF = Cout \oplus Cin$



1. 基本运算部件

1.4 算术逻辑单元ALU





2. 定点数的运算

2.1 移位运算

逻辑移位



左移：高位移出，低位补0
右移：低位移出，高位补0

用于无符号数

算术移位



左移：高位移出，低位补0
右移：低位移出，高位补符号位

用于带符号数（补码）

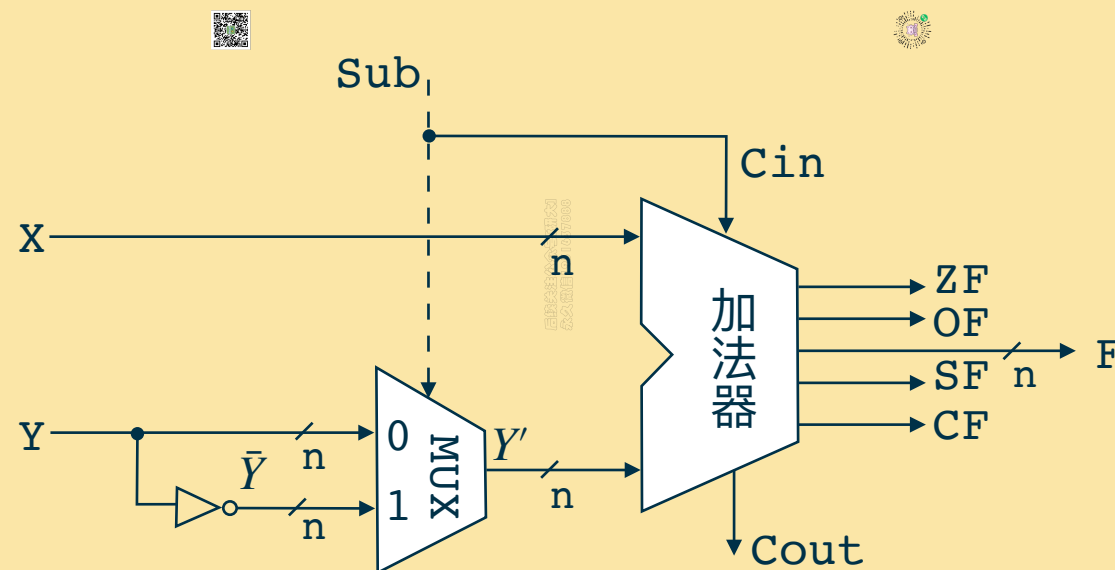
每左移1位，相当于乘2
每右移1位，相当于除以2



2. 定点数的运算

2.2 补码加减运算

①补码加减运算电路



$$[x]_{\text{补}} + [y]_{\text{补}} = [x + y]_{\text{补}}$$

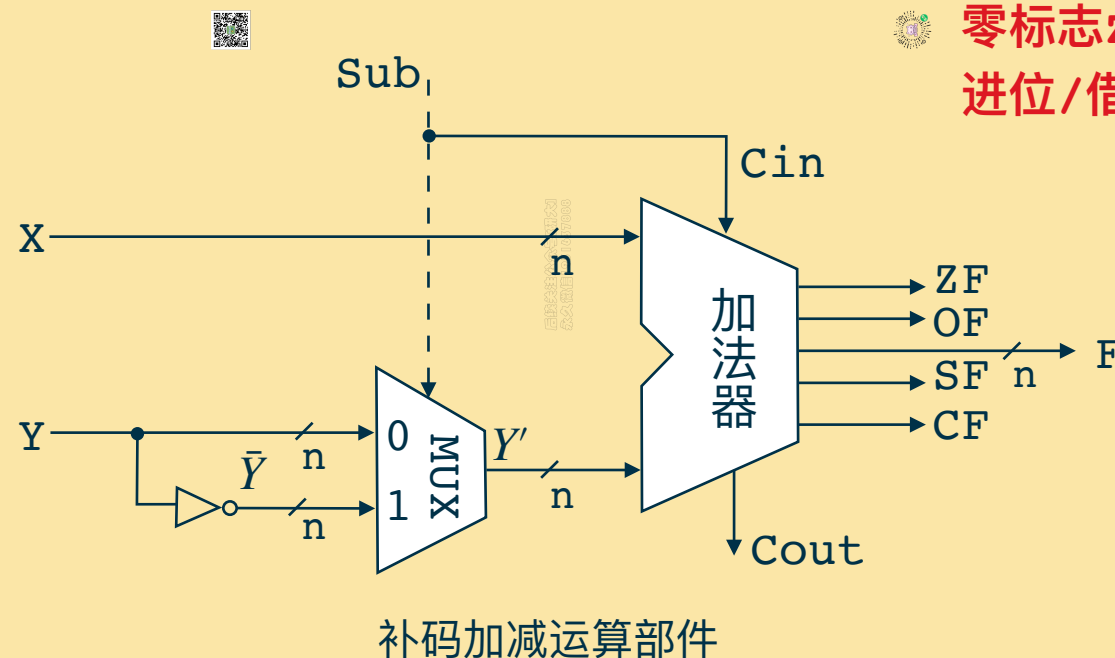
$$[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$



2. 定点数的运算

2.2 补码加减运算

①补码加减运算电路



$$\text{溢出标志OF} = C_n \oplus C_{n-1}$$

$$\text{符号标志SF} = F_{n-1}$$

$$\text{零标志ZF} = 1 \text{ 当且仅当 } F = 0$$

$$\text{进位/借位标志CF} = C_{out} \oplus C_{in}$$

可通过标志信息区分带符号整数运算结果和无符号整数运算结果

SF、OF仅对带符号数有意义，**CF**仅对无符号数有意义



2. 定点数的运算

2.2 补码加减运算



cmp A, B



无符号数看ZF和CF

若**A=B**, 则A-B后ZF=1, CF=0

若**A>B**, 则A-B后ZF=0, CF=0

若**A<B**, 则A-B后ZF=0, CF=1



2. 定点数的运算

2.2 补码加减运算



cmp A, B



带符号数看**SF**、**OF**和**ZF**

若**A=B**, 则A-B后ZF=1

若**A>B**, 则A-B后ZF=0, 且 $SF \oplus OF = 1$

若**A<B**, 则A-B后ZF=0, 且 $SF \oplus OF = 0$



2. 定点数的运算

2.2 补码加减运算

②溢出判断



当且仅当下列4种情况才会发生溢出：

正数+正数=负数

负数+负数=正数

正数-负数=负数

负数-正数=正数



2. 定点数的运算

2.2 补码加减运算

②溢出判断



采用一位符号位

$$V = A_S B_S \overline{S_S} + \overline{A_S} \overline{B_S} S_S$$

$V=0$ ，无溢出

$V=1$ ，有溢出



2. 定点数的运算

2.2 补码加减运算

②溢出判断



采用双位符号位

设置两个符号位 s_{s1} 和 s_{s2} ，最高位 s_{s1} 代表真正的符号

$s_{s1}s_{s2} = 00$: 结果为正数，无溢出

$s_{s1}s_{s2} = 01$: 结果为正溢出

$s_{s1}s_{s2} = 10$: 结果为负溢出

$s_{s1}s_{s2} = 11$: 结果为负数，无溢出



2. 定点数的运算

2.2 补码加减运算

②溢出判断



根据数据位的进位情况判断

若符号位进位 c_s 与最高数位进位 c_1 相同，则无溢出
若符号位进位 c_s 与最高数位进位 c_1 不同，则发生溢出



2. 定点数的运算

2.3 原码乘法运算

①原码一位乘法



0. 1 0 1 1

被乘数 $x=0.x_1x_2x_3x_4=0.1011$

x

0. 1 1 0 1

乘数 $y=0.y_1y_2y_3y_4=0.1101$

这部分叫部分积
乘法的结果就是对部分积进行错位求和

				1	0	1	1		$X \times Y_4 \times 2^{-4}$
			0	0	0	0			$X \times Y_3 \times 2^{-3}$
		1	0	1	1				$X \times Y_2 \times 2^{-2}$
	1	0	1	1					$X \times Y_1 \times 2^{-1}$
0.1	0	0	0	1	1	1	1		

$$X \times Y = \sum_{i=1}^4 (X \times Y_i \times 2^{-i}) = 0.10001111$$



2. 定点数的运算

2.3 原码乘法运算

①原码一位乘法

	0 . 1 0 1 1	被乘数 $X=0.X_1X_2X_3X_4=0.1011$												
x	0 . 1 1 0 1	乘数 $Y=0.Y_1Y_2Y_3Y_4=0.1101$												
<table style="margin: auto; border-collapse: collapse;"> <tr> <td style="border: 1px solid green; padding: 2px 10px;">1 0 1 1</td> <td style="padding: 2px 10px;">.....</td> <td style="padding: 2px 10px;">$X \times Y_4 \times 2^{-4}$</td> </tr> <tr> <td style="border: 1px solid green; padding: 2px 10px;">0 0 0 0</td> <td style="padding: 2px 10px;">.....</td> <td style="padding: 2px 10px;">$X \times Y_3 \times 2^{-3}$</td> </tr> <tr> <td style="border: 1px solid green; padding: 2px 10px;">1 0 1 1</td> <td style="padding: 2px 10px;">.....</td> <td style="padding: 2px 10px;">$X \times Y_2 \times 2^{-2}$</td> </tr> <tr> <td style="border: 1px solid green; padding: 2px 10px;">1 0 1 1</td> <td style="padding: 2px 10px;">.....</td> <td style="padding: 2px 10px;">$X \times Y_1 \times 2^{-1}$</td> </tr> </table>			1 0 1 1		$X \times Y_4 \times 2^{-4}$	0 0 0 0		$X \times Y_3 \times 2^{-3}$	1 0 1 1		$X \times Y_2 \times 2^{-2}$	1 0 1 1		$X \times Y_1 \times 2^{-1}$
1 0 1 1		$X \times Y_4 \times 2^{-4}$												
0 0 0 0		$X \times Y_3 \times 2^{-3}$												
1 0 1 1		$X \times Y_2 \times 2^{-2}$												
1 0 1 1		$X \times Y_1 \times 2^{-1}$												
0 . 1 0 0 0 1 1 1 1														

$$X \times Y = \sum_{i=1}^4 (X \times Y_i \times 2^{-i}) = 0.10001111$$

这部分叫部分积
乘法的结果就是对部分积进行错位求和

1. 用乘数 Y 的每一位依次去乘以被乘数得 $X \times Y_i$ ($i=4, 3, 2, 1$)，若 $Y_i=0$ ，则得0；若 $Y_i=1$ ，则得 X
2. 把1中求得的各项结果 $X \times Y_i$ 在空间上向左错位排列，即逐次左移，可表示为 $X \times Y_i \times 2^{-i}$
3. 对2中求得的结果求和，就是两个无符号数的乘积



2. 定点数的运算

2.3 原码乘法运算

①原码一位乘法

1. 用乘数 Y 的每一位依次去乘以被乘数得 $X \times Y_i$ ($i=4, 3, 2, 1$)，若 $Y_i=0$ ，则得0；若 $Y_i=1$ ，则得 X
2. 把1中求得的各项结果 $X \times Y_i$ 在空间上向左错位排列，即逐次左移，可表示为 $X \times Y_i \times 2^{-i}$
3. 对2中求得的结果求和，就是两个无符号数的乘积



1. 每将乘数 Y 的一位乘以被乘数得 $X \times Y_i$ 后，就将该结果与前面所得的结果累加，得到 P_i （部分积）
2. 在每次求得 $X \times Y_i$ 后，不是将它左移与前次部分积 P_i 相加，而是将部分积 P_i 右移一位与 $X \times Y_i$ 相加
3. 对乘数中为1的位执行加法和右移运算，对为0的位只执行右移运算，而不需执行加法运算



2. 定点数的运算

2.3 原码乘法运算

①原码一位乘法

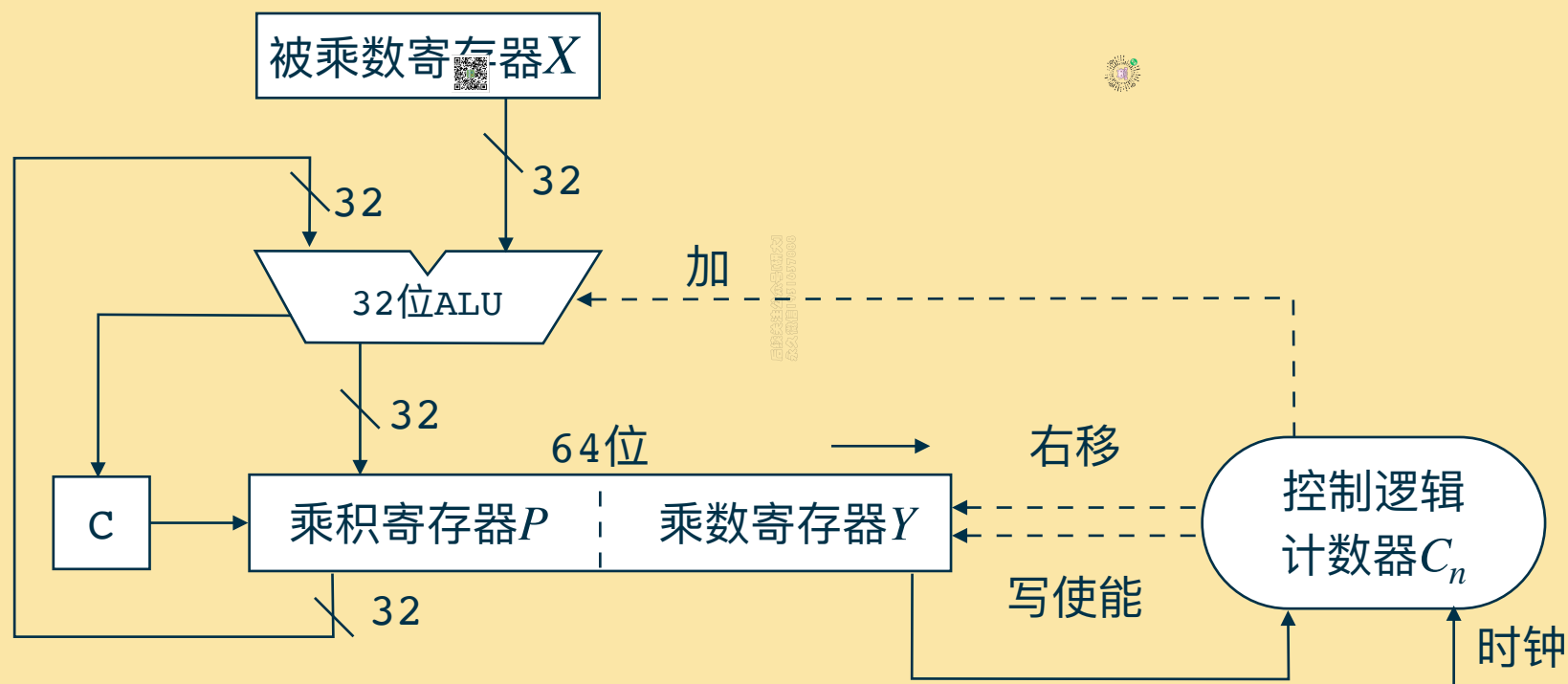
			
	0 . 1 0 1 1		被乘数 $x=0.x_1x_2x_3x_4=0.1011$
x	0 . 1 1 0 1		乘数 $y=0.y_1y_2y_3y_4=0.1101$
	1 0 1 1		$X \times Y_4 \times 2^{-4}$
	0 0 0 0		$X \times Y_3 \times 2^{-3}$
	1 0 1 1		$X \times Y_2 \times 2^{-2}$
	1 0 1 1		$X \times Y_1 \times 2^{-1}$
	0.1 0 0 0 1 1 1 1		

1. 每将乘数 y 的一位乘以被乘数得 $X \times Y_i$ 后，就将该结果与前面所得的结果累加，得到 P_i （部分积）
2. 在每次求得 $X \times Y_i$ 后，不是将它左移与前次部分积 P_i 相加，而是将部分积 P_i 右移一位与 $X \times Y_i$ 相加
3. 对乘数中为1的位执行加法和右移运算，对为0的位只执行右移运算，而不需执行加法运算



2. 定点数的运算

2.3 原码乘法运算



实现32位无符号数乘法运算的逻辑结构图



2. 定点数的运算

2.3 原码乘法运算



【例】已知 $x=0.1101$ ， $y=0.1011$ ，计算 $x \times y$ 的结果

原码乘法运算



【例】已知 $x=0.1101$, $y=0.1011$, 计算 $x \times y$ 的结果

C	P	Y	说明
0	0000	1011	部分积 $P_0=0$
	+1101		
0	1101		$Y_4=1$, 所以+X
0	0110	1101	C、P、Y同时右移1位, 得 P_1
	+1101		
1	0011		$Y_3=1$, 所以+X
0	1001	1110	C、P、Y同时右移1位, 得 P_2



【例】已知 $x=0.1101$, $y=0.1011$, 计算 $x \times y$ 的结果

C	P	Y	说明
0	0000	1011	部分积 $P_0=0$
	+1101		$Y_4=1$, 所以+X
0	1101		
0	0110	1101	C、P、Y同时右移1位, 得 P_1
	+1101		$Y_3=1$, 所以+X
1	0011		
0	1001	1110	C、P、Y同时右移1位, 得 P_2
	+0000		$Y_2=0$, 所以+0 (不做加法)
0	1001		
0	0100	1111	C、P、Y同时右移1位, 得 P_3
	+1101		$Y_1=1$, 所以+X
1	0001		
0	1000	1111	C、P、Y同时右移1位, 得 P_4

最终P和Y拼起来就是
8位乘积



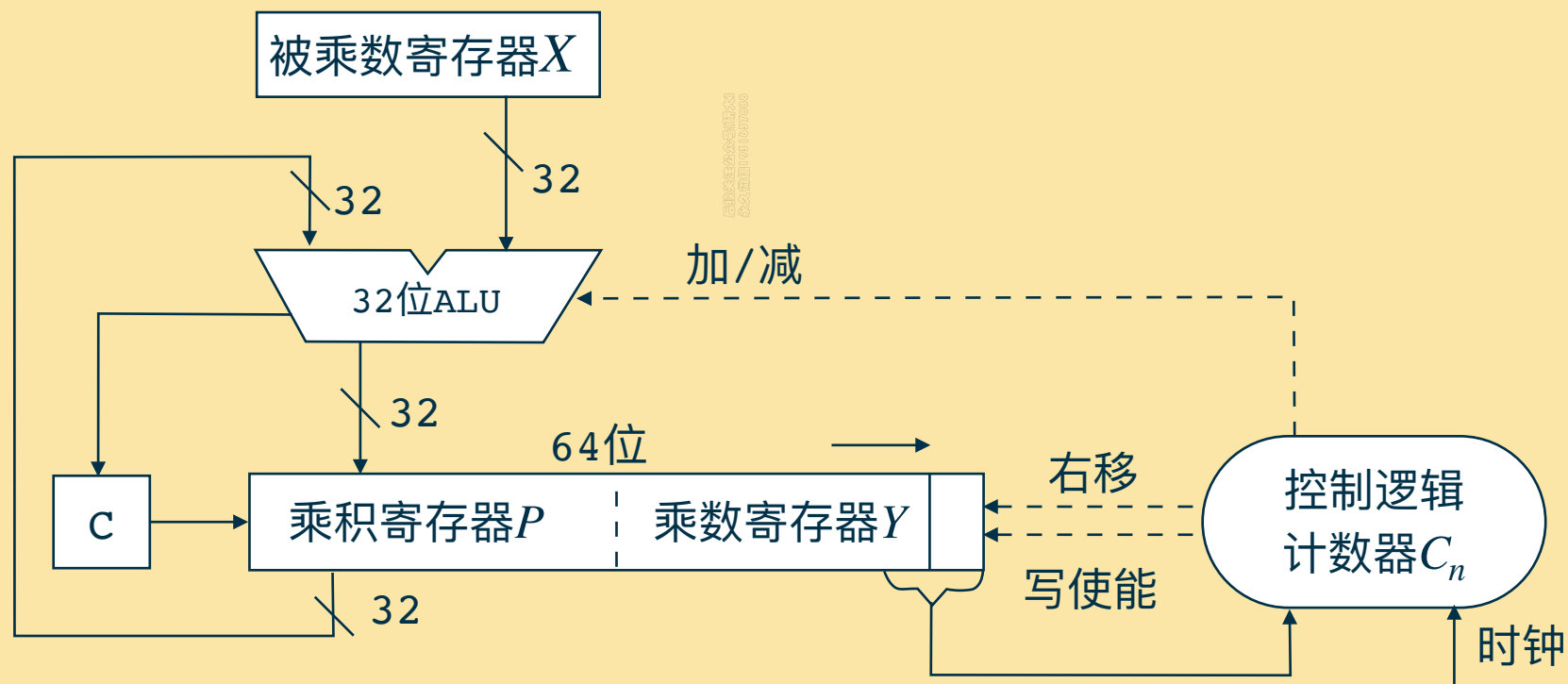
2. 定点数的运算

2.4 补码乘法运算

Booth算法的原理推导比较逆天💧



这里仅介绍计算方法



实现补码一位乘法的逻辑结构图



2. 定点数的运算

2.4 补码乘法运算

 Booth算法的原理推导比较逆天😓
这里仅介绍计算方法

- (1) 乘数最低位增加一位辅助位 $Y_{-1}=0$
- (2) 根据 Y_iY_{i-1} 的值，决定是 $+[X]_{\text{补}}$ 、 $-[X]_{\text{补}}$ 还是 $+0$
- (3) 每次加减后，算数右移1位，得到部分积
- (4) 重复 (2) (3) n 次，得到结果

Y_iY_{i-1}	操作
00	0
01	$+[X]_{\text{补}}$
10	$+[-X]_{\text{补}}$
11	0

$Y_i Y_{i-1}$	操作
00	0
01	$+ [X]_{\text{补}}$
10	$+ [-X]_{\text{补}}$
11	0



【例】已知 $x=1101$, $y=0110$, 计算 $x \times y$ 的结果

$[-x]_{\text{补}}=0011$, $Y=[y]_{\text{补}}=0110$

P	Y	Y_{-1}	说明
0000	0110	0	设 $Y_{-1}=0$, $[P_0]_{\text{补}}=0$
0000	0011	0	$Y_0 Y_{-1}=00$, P、Y直接右移1位得 $[P_1]_{\text{补}}$
+0011 0011 0001	1001	1	$Y_1 Y_0=10$, $+ [-x]_{\text{补}}$ P、Y同时右移1位, 得 $[P_2]_{\text{补}}$
0000	1100	1	$Y_2 Y_1=11$, P、Y直接右移1位得 $[P_3]_{\text{补}}$
+1101 1101 1110	1110		$Y_3 Y_2=01$, $+ [x]_{\text{补}}$ P、Y同时右移1位, 得 $[P_4]_{\text{补}}$

最终P和Y拼起来就是8位乘积



2. 定点数的运算

2.5 原码除法运算

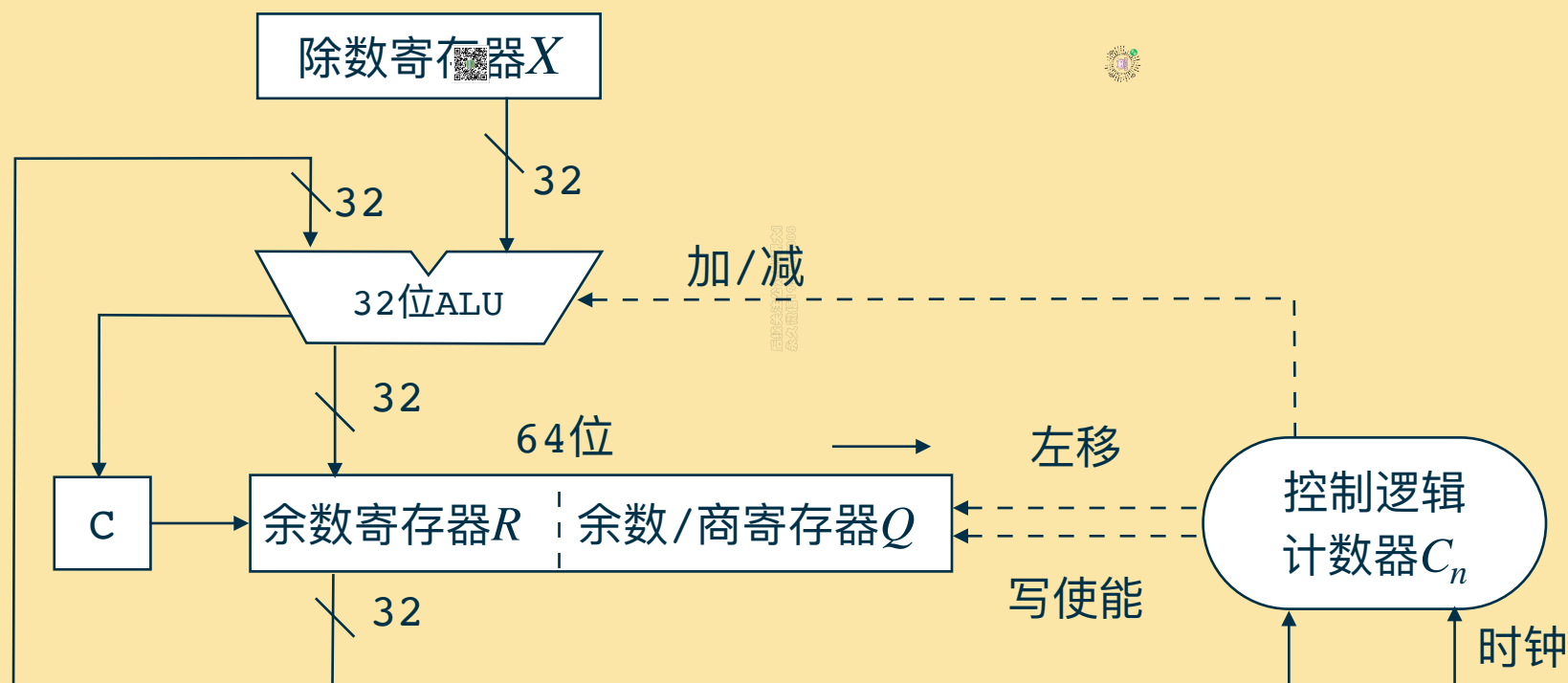
除数	1 0 1 1	1 0 0 1 1 1 0 1	商	1 1 1 0
		1 0 1 1	被除数	
		1 0 0 0 1		
		1 0 1 1		
		0 1 1 0 0		
		1 0 1 1		
		0 0 1 1		
		0 0 0 0		
		0 0 1 1		
		0 0 1 1	余数	

- (1) 被除数和除数相减，若够减，则上商为1，否则上0
- (2) 每次得到的差为中间余数，将除数右移后与上次的中间余数比较，若够减，则上商为1，否则上0
- (3) 重复执行 (2)，直到求得的商的位数足够为止



2. 定点数的运算

2.5 原码除法运算



32位除法运算逻辑结构



2. 定点数的运算

2.5 原码除法运算

左移 = 下拉一位

$$\begin{array}{r} 1 \\ 11 \overline{) 157} \\ \underline{11} \\ 47 \end{array}$$

$11 < 15$, 够减, 商1

此时应该把7拉下来, 变成47

而 $47 = 4 \times 10 + 7$

也就是说, 下拉一位的本质是将当前的中间余数扩大1倍, 再加下一位

而左移1位=乘以2 (扩大1倍), 所以计算机的左移行为就是在下拉数字



2. 定点数的运算

2.5 原码除法运算

除数	1 0 1 1	1 1 1 0 1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 0 1 1 0 1 1 0 1 1 0 0 1 0 1 1 0 0 1 1 0 0 0 0 0 0 1 1	商
		1 0 0 1 1 1 0 1	被除数
		1 0 1 1	
		1 0 0 0 1	
		1 0 1 1	
		0 1 1 0 0	
		1 0 1 1	
		0 0 1 1	
		0 0 0 0	
		0 0 1 1	余数

中间余数

初始化：R/Q放被除数，y放除数

重复n次：

(1) R和Q整体左移1位

(2) $R = R - Y$

(3) 如果 $R \geq 0$ ：商1

如果 $R < 0$ ：商0， $R = R + Y$

最后：

Q中是商

R中是余数



2. 定点数的运算

2.5 原码除法运算

$2n$ 位被除数 $\div$ n 位除数 $\rightarrow$ n 位商 $+$ n 位余数

两个 n 位定点正整数相除

高位补0

两个 n 位定点正小数相除

低位补0

【例】已知 $x=0.1011$, $y=1.1101$, 计算 x/y 的结果



余数寄存器R

余数/商寄存器Q

说明

01011	0000	开始 $R_0=X$
+10011		$R_1=X-Y$
11110	00000	$R_1<0$, 故 $Q_4=0$
+01101		恢复余数: $R_1=R_1+Y$
01001		得 R_1
10110	0000	$2R_1$ (R和Q同时左移, 空出一位商)
+10011		$R_2=2R_1-Y$
01001	00001	$R_2>0$, 故 $Q_3=1$
10010	0001	$2R_2$ (R和Q同时左移, 空出一位商)
+10011		$R_3=2R_2-Y$
00101	00011	$R_3>0$, 故 $Q_2=1$
01010	0011	$2R_3$ (R和Q同时左移, 空出一位商)
+10011		$R_4=2R_3-Y$
11101	00110	$R_4>0$, 故 $Q_1=1$
+01101		恢复余数: $R_4=R_4+Y$
01010	00110	得 R_4
10100	0110	$2R_4$ (R和Q同时左移, 空出一位商)
+10011		$R_5=2R_4-Y$
00111	01101	$R_5>0$, 故 $Q_0=1$